

Refonte PIN Employe — Specs Backend

Holy Fork | 23 juillet 2026

Probleme actuel

- Le PIN employe est **auto-generé par le frontend** à la creation du compte (`Math.random()`)
- Le PIN est affiche **une seule fois** apres la creation — si le manager ferme le dialog, le PIN est perdu
- L'employe **ne choisit jamais** son propre PIN
- Aucun moyen de retrouver, changer ou reinitialiser le PIN apres la creation
- Un employe sans PIN est **bloqué** sur l'ecran device login (bouton grise)

Nouveau systeme

- L'employe **definit lui-meme son PIN** lors de sa premiere connexion sur la tablette
- Le PIN est cree avec **double saisie** (confirmation) directement sur le device
- Le manager peut voir le **statut du PIN** (actif/non defini) et le **reinitialiser** depuis la fiche employe
- L'employe peut **changer son PIN** depuis son profil
- Plus de generation aleatoire cote frontend

1. Nouveau flow device login

Tablette en salle – /device

Etape 1 : PIN Restaurant (inchange)

```
POST /api/auth/device-login/  
{ restaurant_id, pin_restaurant }  
→ deviceToken
```

Etape 2 : Selection employe (inchange)

```
GET /api/auth/restaurant-employees/  
→ liste employes avec has_pin: true/false
```

Etape 3 : Selon has_pin

```
SI has_pin == true :  
→ Ecran "Entrez votre PIN" (inchange)  
POST /api/auth/quick-login/
```

```
{ device_token, pin_code }
→ JWT
```

```
SI has_pin == false :                               ← NOUVEAU
→ Ecran "Definir votre code PIN"
L'employe tape 4 chiffres + confirme
POST /api/auth/set-pin/                             ← NOUVEL ENDPOINT
{ device_token, employee_id, pin_code }
→ JWT (login automatique apres definition)
```

2. Endpoints a creer / modifier

Endpoint 1 : Definir son PIN NOUVEAU

Route	POST /api/auth/set-pin/
Authentification	Aucune (meme principe que device-login : le <code>device_token</code> fait office de preuve)
Rate-limiting	5/min par IP + <code>device_token</code>

Request :

```
{
  "device_token": "abc123...",
  "employee_id": 42,
  "pin_code": "1031"
}
```

Validations backend :

1. Le `device_token` est valide et non expire
2. L'`employee_id` appartient au restaurant associe au `device_token`
3. L'employe n'a **pas deja un PIN** (`has_pin == false`). Si un PIN existe deja → 400 "PIN deja defini"
4. Le `pin_code` fait exactement 4 chiffres (`^\d{4}$`)

Responses :

```
// 200 – Succes (meme format que quick-login → login auto)
{
  "message": "PIN defini avec succes",
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "user_id": 15,
  "employee_id": 42,
  "employee_name": "Nicolas ROUVROY",
  "employee_type": "Manager Salle"
}

// 400 – PIN deja defini
```

```
{ "detail": "Cet employe a deja un PIN. Utilisez change-pin pour le modifier." }

// 400 – PIN invalide
{ "detail": "Le PIN doit contenir exactement 4 chiffres." }

// 401 – Device token invalide
{ "detail": "Session appareil expiree." }

// 404 – Employe non trouve dans ce restaurant
{ "detail": "Employe introuvable." }
```

Logique backend :

```
class SetPinView(APIView):
    throttle_classes = [DeviceLoginThrottle]

    def post(self, request):
        device_token = request.data.get('device_token')
        employee_id = request.data.get('employee_id')
        pin_code = request.data.get('pin_code')

        # 1. Valider device_token → recuperer restaurant
        restaurant = validate_device_token(device_token)
        if not restaurant:
            return Response({"detail": "Session appareil expiree."}, 401)

        # 2. Verifier que l'employe appartient au restaurant
        employee = get_employee_for_restaurant(employee_id, restaurant)
        if not employee:
            return Response({"detail": "Employe introuvable."}, 404)

        # 3. Verifier qu'il n'a pas deja un PIN
        if employee.user and employee.user.pin_code:
            return Response({"detail": "PIN deja defini."}, 400)

        # 4. Valider format PIN
        if not re.match(r'^\d{4}$', pin_code):
            return Response({"detail": "Le PIN doit contenir 4 chiffres."}, 400)

        # 5. Sauvegarder le PIN + generer JWT
        employee.user.pin_code = pin_code # ou hash si souhaite
        employee.user.save()

        tokens = generate_jwt_for_user(employee.user)
        return Response({
            "message": "PIN defini avec succes",
            "access_token": tokens.access,
            "refresh_token": tokens.refresh,
            ...
        })
```

Endpoint 2 : Changer son PIN NOUVEAU

Route	
	POST /api/auth/change-pin/

Authentification	JWT requis (employe connecte)
Rate-limiting	5/min par user

Request :

```
{
  "old_pin_code": "1031",
  "new_pin_code": "5678"
}
```

Validations :

1. L'utilisateur connecte a un PIN existant
2. `old_pin_code` correspond au PIN actuel (comparaison constante : `hmac.compare_digest`)
3. `new_pin_code` fait 4 chiffres, different de l'ancien

Responses :

```
// 200
{ "message": "PIN modifie avec succes." }

// 400 – Ancien PIN incorrect
{ "detail": "PIN actuel incorrect." }

// 400 – Meme PIN
{ "detail": "Le nouveau PIN doit etre different de l'ancien." }
```

Endpoint 3 : Reset PIN par un admin NOUVEAU

Route	POST <code>/api/employees/{id}/reset-pin/</code>
Authentification	JWT requis, permission manager/admin

Request :

```
{}
```

 // Pas de body – le reset supprime le PIN**Logique :**

1. Mettre le `pin_code` de l'employe a `null` / vide
2. L'employe redevient `has_pin: false`
3. A sa prochaine connexion device, il repassera par le flow "Definir votre PIN"

Responses :

```
// 200
{ "message": "PIN reinitialise. L'employe devra definir un nouveau PIN." }
```

```
// 403 – Pas les droits
{ "detail": "Permission refusee." }
```

Modification : Register / Invite User MODIFIER

Le champ `pin_code` ne doit **plus etre obligatoire** a la creation d'un utilisateur :

```
# Avant (actuel)
class UserRegistrationSerializer:
    pin_code = serializers.CharField(required=True, min_length=4, max_length=4)

# Apres
class UserRegistrationSerializer:
    pin_code = serializers.CharField(required=False, allow_null=True,
                                    allow_blank=True, min_length=4, max_length=4)
```

Si `pin_code` n'est pas fourni → l'utilisateur est cree avec `has_pin = false` . Il definira son PIN lui-meme sur le device.

Modification : GET restaurant-employees MODIFIER

Le champ `has_pin` existe deja dans la reponse — **rien a changer**. Verifier juste qu'il reflete bien la realite (`has_pin = bool(user.pin_code)`).

3. Resume des changements backend

Type	Endpoint / Fichier	Effort
NOUVEAU	POST <code>/api/auth/set-pin/</code>	~30 min
NOUVEAU	POST <code>/api/auth/change-pin/</code>	~20 min
NOUVEAU	POST <code>/api/employees/{id}/reset-pin/</code>	~15 min
MODIFIER	<code>UserRegistrationSerializer</code> — <code>pin_code</code> optionnel	~5 min
SUPPRIMER	Le frontend n'enverra plus de <code>pin_code</code> a la creation	Rien cote backend

Effort total backend estime : ~1h15

4. Ce que le frontend fera de son cote

Pas d'action requise cote backend pour ces points — juste pour information :

- **Nouvel ecran "Definir votre PIN"** sur `/device` quand `has_pin == false` — double saisie (PIN + confirmation) puis appel `POST /api/auth/set-pin/`

- **Fiche employe** (admin) : afficher statut PIN (actif / non defini) + bouton "Reinitialiser le PIN" qui appelle `POST /api/employees/{id}/reset-pin/`
- **Profil employe** : bouton "Modifier mon PIN" qui appelle `POST /api/auth/change-pin/`
- **Suppression de `generatePin()`** dans `register.tsx` et `employe-detail.tsx` — le frontend n'enverra plus de `pin_code` a la creation

5. Ordre d'implementation suggere

1. **Backend** : rendre `pin_code` optionnel dans le serializer registration
2. **Backend** : creer `POST /api/auth/set-pin/`
3. **Frontend** : supprimer `generatePin()` , creer l'ecran "Definir votre PIN"
4. **Backend** : creer `POST /api/employees/{id}/reset-pin/`
5. **Frontend** : ajouter bouton reset sur la fiche employe
6. **Backend** : creer `POST /api/auth/change-pin/`
7. **Frontend** : ajouter "Modifier mon PIN" dans le profil

Document genere dans le cadre de l'audit testing frontend Holy Fork. Contact : equipe frontend.